

MSE 381

**Final Project - Design of a Gain-Scheduled
Position Control System for an Automatic Door**

Group 3:

Ahmed Al-Gattan	
Jessica Gong	
Ryan Thomas	

Table of Contents

Abstract.....	3
Introduction.....	3
Theory.....	4
Modelling.....	4
Linearization.....	4
Results.....	5
1. Nonlinear System Modelling & Simulation.....	5
Model Implementation in Simulink.....	5
Integration to Obtain Velocity and Position.....	6
Nonlinear Spring Term kx^3	6
Implementation in Simulink.....	7
Transfer Function Derivation.....	7
Simulink Model Structure.....	8
2. PID Controller Design, Simulation & Analysis.....	13
3. Intro to Gain-Scheduled PID Controller.....	23
4. Gain-Scheduled PID Controller Design, Simulation & Analysis.....	25
Discussion.....	29
Conclusion.....	29
Appendix.....	30

Abstract

This project presents the modelling, control design, and simulation of a nonlinear position control system for a spring-loaded automatic door. The system features a nonlinear restoring force defined by $F = kx^3$, viscous damping, and a force actuator as the control input. The objective is to design and evaluate a gain-scheduled PID controller capable of accurately and efficiently tracking a reference signal across a range of door positions. First, the nonlinear system is modelled and linearized at multiple operating points to assess the validity of the linear approximation. A conventional PID controller is then tuned for the mid-range operating point and evaluated against performance criteria including settling time, overshoot, and steady-state error. To improve performance across the full range of motion, a gain-scheduled PID controller is implemented with region-specific gains. Finally, actuator saturation limits are introduced to test real-world feasibility. Simulation results demonstrate the effectiveness of gain scheduling in enhancing system responsiveness and robustness compared to a single PID controller.

Introduction

Automatic doors are widely used in commercial and industrial settings, where reliability, speed, and safety are critical. In many of these systems, control design must account for nonlinear dynamics, actuator constraints, and safety mechanisms in the event of power failure. This project focuses on the position control of a spring-loaded automatic sliding door, where a nonlinear spring mechanism ensures the door closes automatically if power is lost. The system's behavior is governed by a nonlinear force-displacement relationship $F = kx^3$, making it an ideal candidate for advanced control strategies.

The main goal of this project is to model, simulate, and control the door's position using both conventional and gain-scheduled PID controllers. While a standard PID controller can offer acceptable performance near a specific operating point, its effectiveness often diminishes in systems with strong nonlinear characteristics. Gain scheduling allows the controller to adapt to different operating regions, improving accuracy and responsiveness across the full range of motion.

This report details the modelling and linearization of the system, the design and tuning of PID controllers, and the comparative performance analysis under different control strategies. The project also considers practical constraints such as actuator saturation and evaluates the robustness of the designed control systems under such limitations.

Theory

Modelling

The system under study is a spring-loaded automatic door with position control, where the primary objective is to regulate the door's displacement through force input generated by an actuator. The system exhibits nonlinear behaviour due to the inclusion of a nonlinear spring with a force-displacement relationship $F = kx^3$, intended to ensure passive closure in case of actuator failure.

The equation governing the motion of the door is derived using Newton's second law, considering mass (m), the viscous damping coefficient (c), and the nonlinear spring stiffness constant (k), resulting in a second-order differential equation:

$$m\ddot{x} + c\dot{x} + kx^3 = F$$

Where x is position, $\dot{x}$ is velocity, $\ddot{x}$ is acceleration, and F is input force.

Linearization

In order to simplify the controller design, the nonlinear system is linearized around a specific point using a first-order Taylor series expansion, resulting in:

$$m\ddot{\hat{x}} + c\dot{\hat{x}} + 3kx_0^2\hat{x} = \hat{F}$$

Where $\hat{x} = x - x_0$ and $\hat{F} = F - F_0$

This is a linear time-invariant second-order system where the spring constant has been replaced by $3kx_0^2$, which means that the system stiffness increases with displacement. To support gain scheduling, linearization can be performed at multiple points.

PID control and Gain scheduling

$$F = k_p e + k_i \int e dt + k_d \dot{x}$$

Due to the nonlinear behaviour of the system, a single set of PID gains may not be sufficient for optimal performance across the full operating range. Thus, gain scheduling is used, where the PID gains are adapted based on the current door position. The position domain is divided into regions, with a separate PID tuning for each region to improve stability, response speed, and overshoot control.

Results

1. Nonlinear System Modelling & Simulation

Simulate the open-loop system response for the following two conditions in SIMULINK:

a) Simulate the original nonlinear system with the nonlinear parameters. The initial condition is considered zero for consistency.

The open-loop response was simulated under two conditions:

1. The original nonlinear system.
2. The nonlinear system was linearized at three different operating points.

The initial condition was set to zero for consistency. The main objective was to obtain the door position response over time, which is essential for creating the necessary comparison graphs.

The nonlinear equation of motion for the system is:

$$m\ddot{x} + c\dot{x} + kx^3 = F$$

Rearranging to isolate the acceleration term:

$$\ddot{x} = \frac{F - c\dot{x} - kx^3}{m}$$

Model Implementation in Simulink

The term $F - c\dot{x} - kx^3$ represents the net force on the system and was implemented in Simulink using a Sum block:

- The + input corresponds to the external force F.
- The two - inputs correspond to the damping rate $c\dot{x}$ and the nonlinear spring term kx^3 .

A Step block was used to generate F, which connects directly to the Sum block. The output of the Sum block is multiplied by $1/m$ using a Gain block to calculate the acceleration $\ddot{x}$.

Integration to Obtain Velocity and Position

Acceleration is integrated twice:

1. The first Integrator outputs the velocity $\dot{x}$, which is also multiplied by c (using a Gain block) to form the damping term for feedback.
2. The second Integrator outputs the position x , which is used to calculate the nonlinear spring term kx^3 .

Nonlinear Spring Term kx^3

The position x from the second Integrator is sent to a math Function block, which computes x^3 directly. This approach provides flexibility, as the nonlinear term can be easily modified inside the function if needed. The output of the math Function block is multiplied by the spring constant k using a Gain block to produce kx^3 . This value is then fed back into the Sum block's second negative input.

The model is then completed, which is shown below in Figure 1.

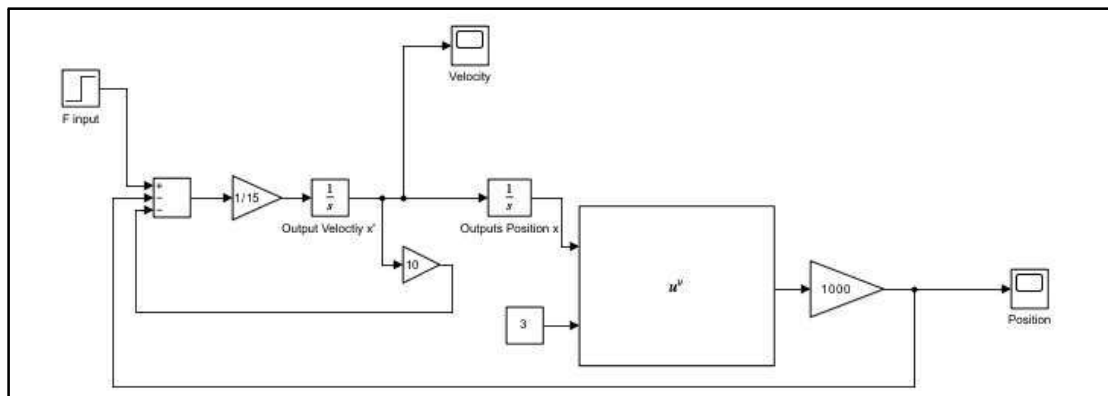


Figure 1: Overall Simulink Model

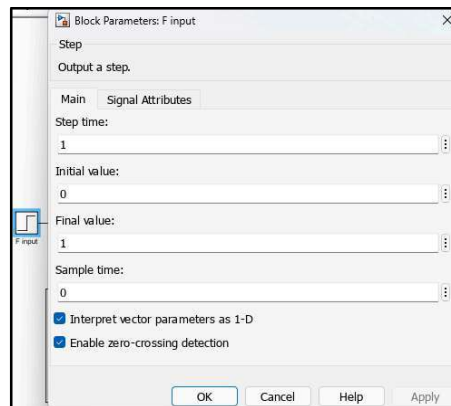


Figure 2: Force Input Values

b) Linearize the system with three desired values as shown in Figure 3 ($x_0 = 0$, $x_0 = 0.5$, $x_0 = 1$). Simulate the new linearized system in SIMULINK (Use the same initial conditions used in part a.)

The linearized model was simulated in Simulink for three operating points: $x_0=0$, $x_0=0.5$, and $x_0=1$, using the same initial conditions as in Part (a). The objective, as in the nonlinear case, was to obtain the position vs. time response.

The linearization is based on the following relationships:

$$\begin{aligned}\hat{F} &= F - F_0 \\ \hat{x} &= x - x_0 \\ F_0 &= kx_0^3\end{aligned}$$

Here, F_0 is the constant spring force at the chosen operating point x_0 , and $\hat{F}$ and $\hat{x}$ are the deviations of force and position from the operating point values.

Implementation in Simulink

A **Step** block was used to generate the input force F . To obtain $\hat{F}$, a **Sum** block was configured to subtract F_0 from F , where F_0 was provided by a **Constant** block. The constant value F_0 was calculated for each x_0 using:

$$F_0 = kx_0^3$$

With $k=1000$, the values were:

- $F_0 = 0$ for $x_0=0$
- $F_0 = 125$ for $x_0= 0.5$
- $F_0 = 1000$ for $x_0=1$

Transfer Function Derivation

From the equation of motion:

$$m\ddot{x} + c\dot{x} + 3kx_0^2\hat{x} = \hat{F}$$

Transfer function:

$$\frac{\hat{x}(s)}{\hat{F}(s)} = \frac{1}{ms^2 + cs + 3kx_0^2}$$

$m=15\text{kg}$, $c=10\text{N}\cdot\text{s}/\text{m}$, and $k=1000\text{ N}/\text{m}^3$

- For $x_0=0$

$$\frac{\hat{x}(s)}{\hat{F}(s)} = \frac{1}{15s^2 + 10s}$$

- For $x_0=0.5$

$$\frac{\hat{x}(s)}{\hat{F}(s)} = \frac{1}{15s^2 + 10s + 750}$$

- For $x_0=1$

$$\frac{\hat{x}(s)}{\hat{F}(s)} = \frac{1}{15s^2 + 10s + 3000}$$

Simulink Model Structure

Each transfer function was implemented using a Transfer Function block. The output of the transfer function corresponds to $\hat{x}$. To recover the total position x , a second **Sum** block was used to add $\hat{x}$ to x_0 , where x_0 was provided by a Constant block.

The complete Simulink model was built with three parallel branches, one for each x_0 value, so that the results could be compared side-by-side. The model for $x_0=0$ is shown in Figure 2, with similar configurations for $x_0=0.5$ and $x_0=1$.

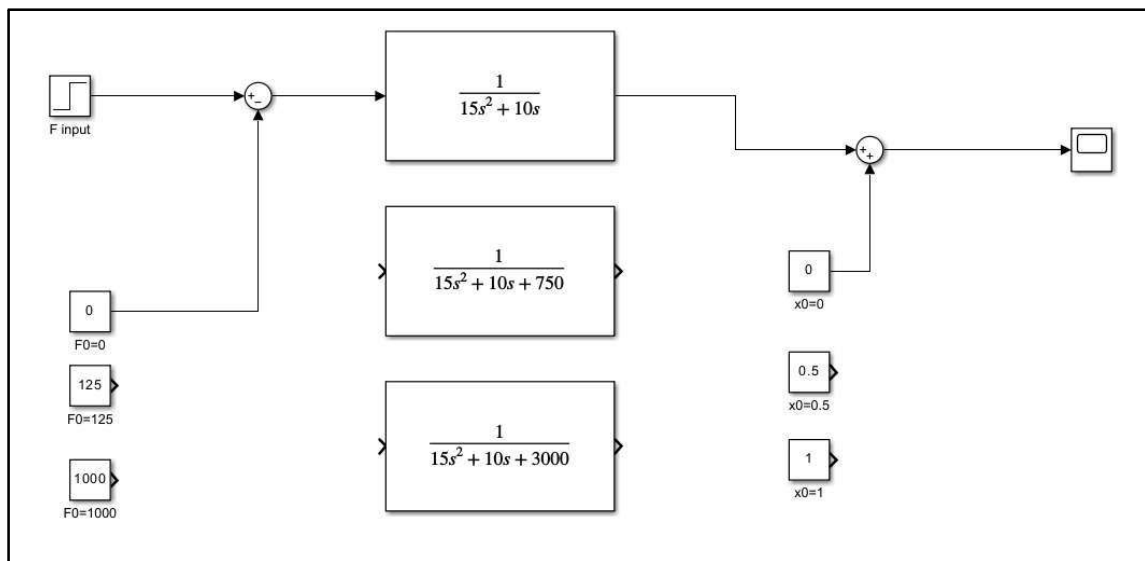


Figure 3: Linearized System

c) Plot your SIMULINK results $x - t$ for the above two cases. Then, compare them and explain how accurate your linearization is.

- From the above model the position graphs were generated, one plot from the non-linearized model and 3 from the linearized model, since it has 3 different x_0 .

For part a)

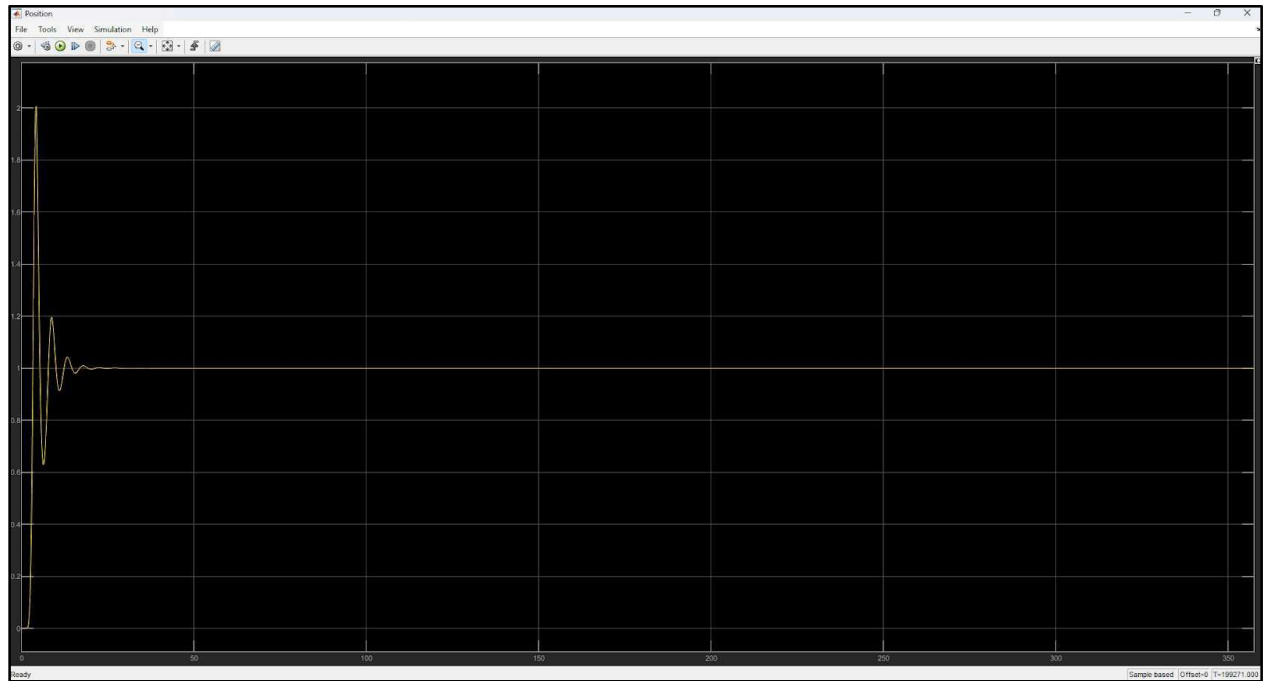


Figure 4: Position Plot for Nonlinear System

Part b)

For $x_0=0$ we get $k=0$ and $F_0=0$

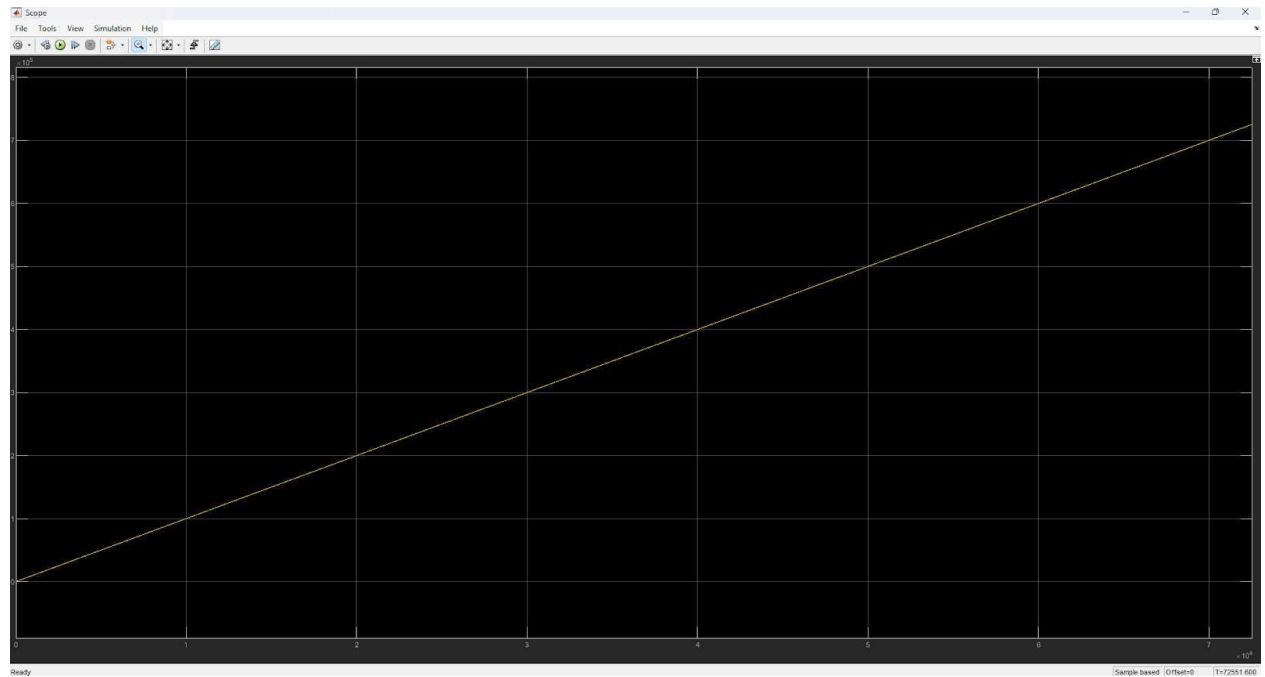


Figure 5: Position Plot for Linear System $x_0=0$

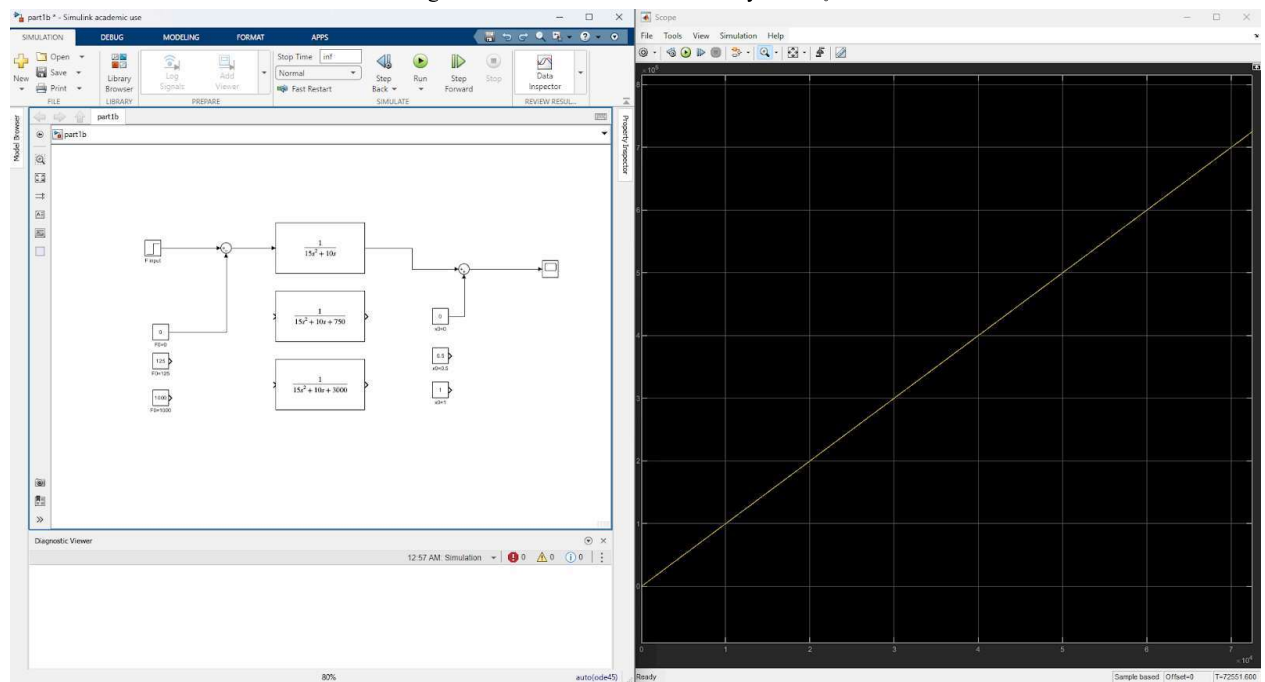


Figure 5: K and x_0 Values 1st Position

For $x_0=0.5$, the value for $K=750$, and $F_0=125$

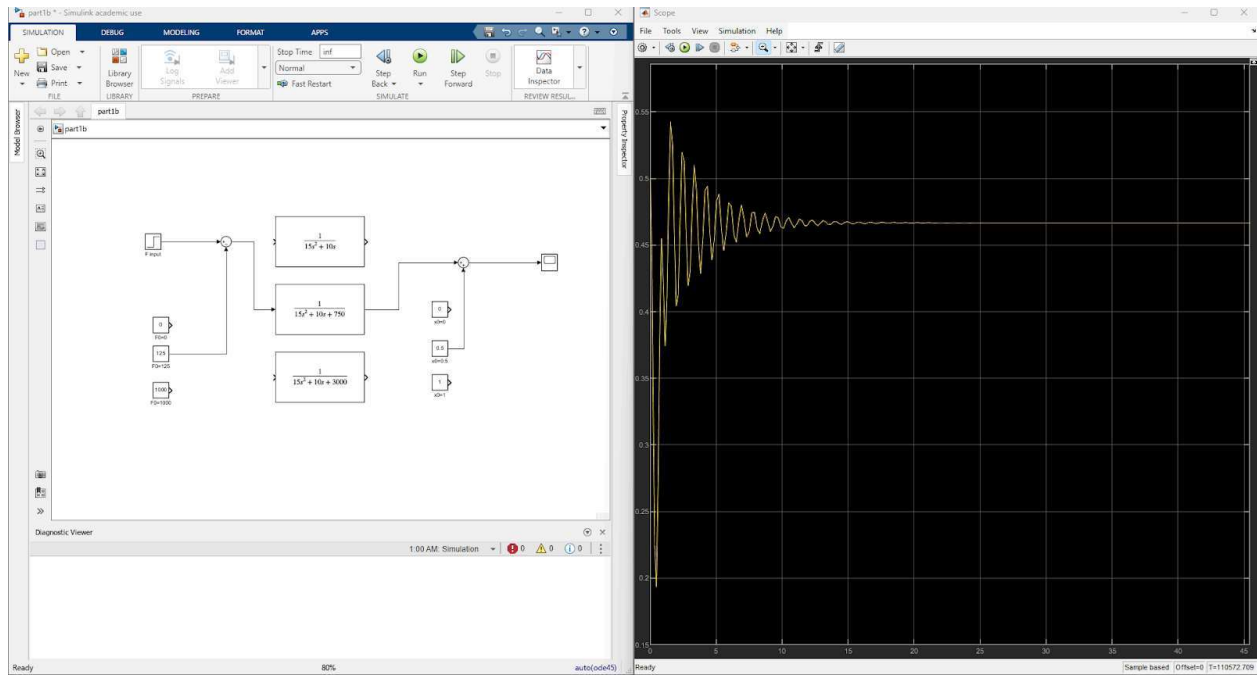


Figure 6: K and x_0 Values 2nd Position

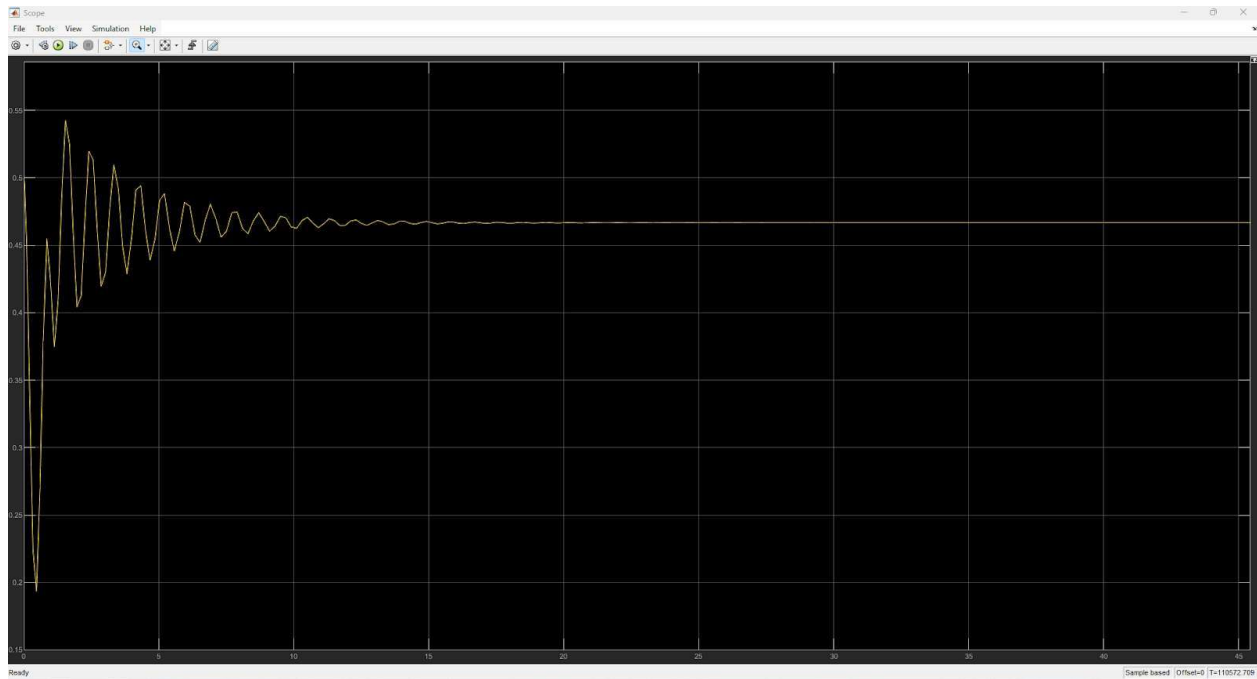


Figure 7: Position Plot for Linear System $x_0=0.5$

For $x_0=1$, the value for $K=3000$, and $F_0=1000$

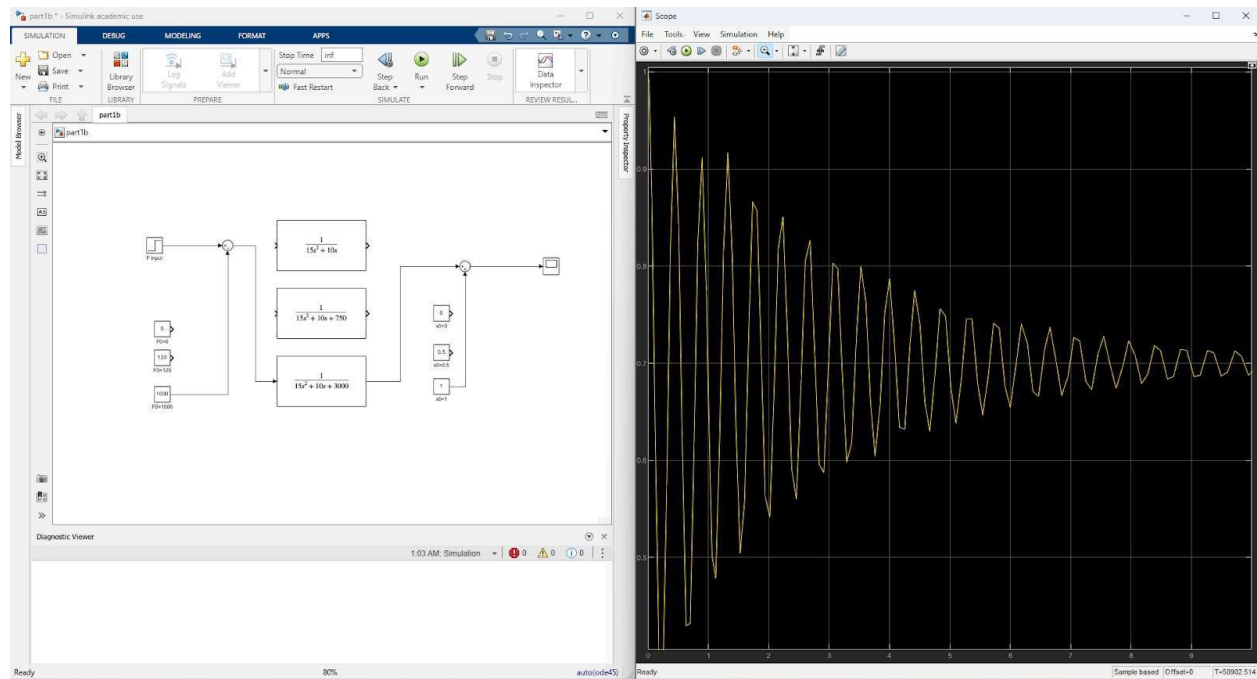


Figure 8: K and x_0 Values 3rd Position

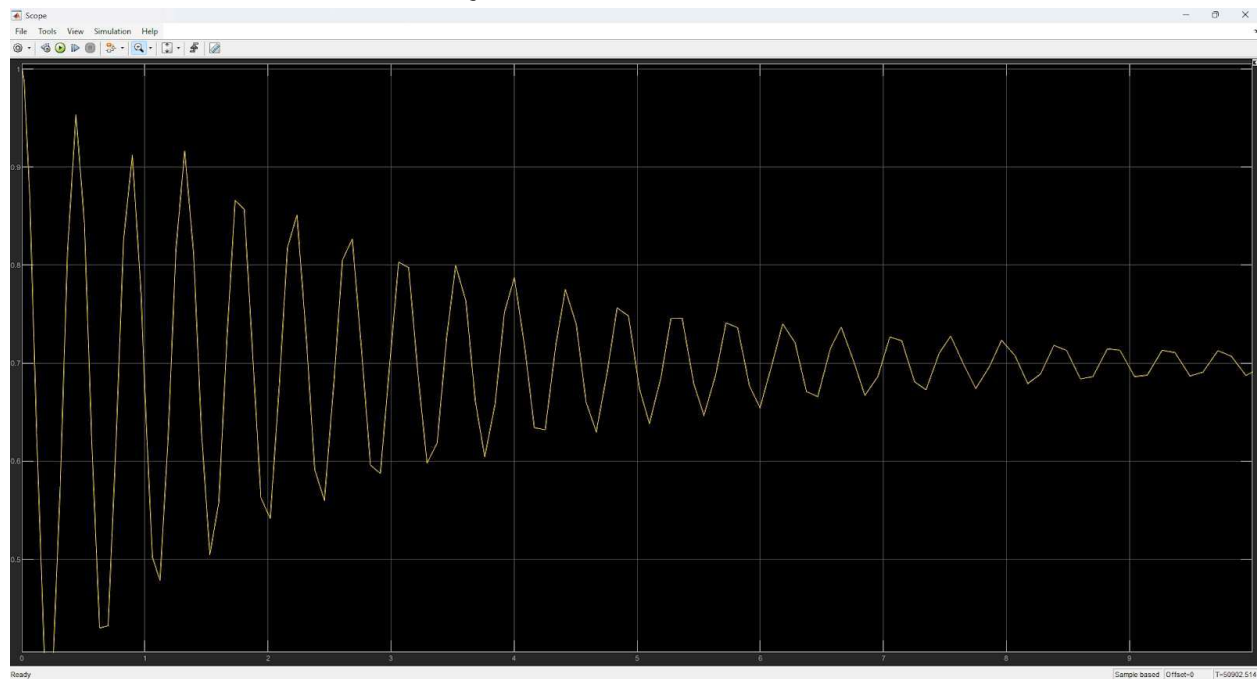


Figure 9: Position Plot for Linear System $x_0=1$

When comparing all four graphs from parts (a) and (b), the results generally reflect the expected linearized behaviour, as most cases eventually reach a steady state. The primary reason the first graph ($x_0=0$) did not exhibit oscillations is likely due to the initial operating point being zero, which results in minimal nonlinear spring influence. As x_0 increases, the nonlinear spring effect becomes stronger, leading to more pronounced oscillations in the system's response.

2. PID Controller Design, Simulation & Analysis

a) Linearize the nonlinear system for $x_0 = 0.5$ m using equations (2) and (3).

- To linearize the system, an operation point must be chosen around which the Taylor expansion can be applied. The given operation point is $x_0=0.5$ m, which corresponds to the door being halfway open.

Using first-order Taylor expansion for the nonlinear x^3 term, we obtain:

$$x^3 = 0.5^3 + 3(0.5^2)(x - 0.5) = 0.125 + 0.75(x - 0.5)$$

$$Kx^3 = 1000 [0.125 + 0.75(x - 0.5)] = 125 + 750(x - 0.5)$$

$$15\ddot{x} + 10\dot{x} + 750x = F(t)$$

$$TF = \frac{X(s)}{F(s)} = \frac{1}{15s^2 + 10s + 750}$$

b) Design a PID controller for the nonlinear system in an operating range of $0 \leq x \leq 1$. To investigate the performance of the controller, the closed-loop control system should precisely follow the reference signal shown in Figure 3 (x_{ref}).

To test and design a PID controller for the nonlinear system, we first created the nonlinear model in Simulink.

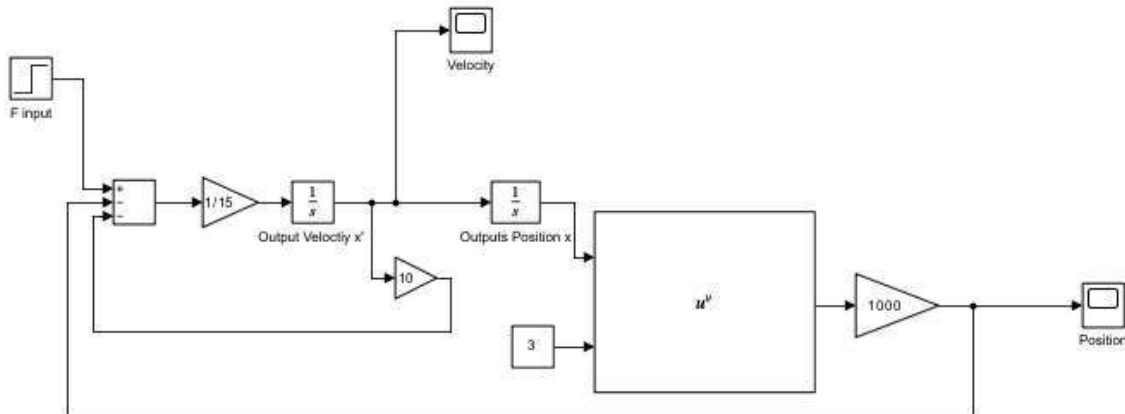


Figure 10: Nonlinear System from Part 1

The setup is very similar to part a of 1, the only difference change was done to to simulate and analyze the controlled system, we used the reference signal provided in the project manual. This signal was implemented in Simulink using the Repeating Sequence Xref block, which was configured to generate the specified position steps over time. The generated reference signal is shown below:

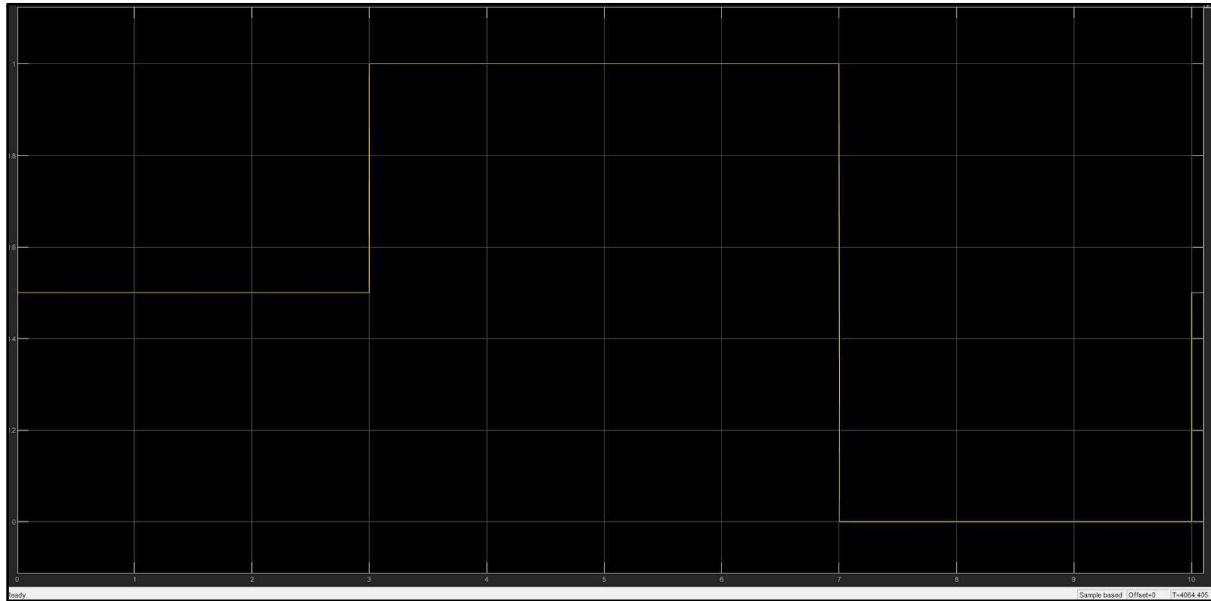


Figure 11: Reference Signal

Now, we can start using the PID controller to get our system to match the reference signal. The completed simulink block diagram is shown below, along with the output for a baseline gain setting of $K_p=220$, $K_i=600$, and $K_d=140$.

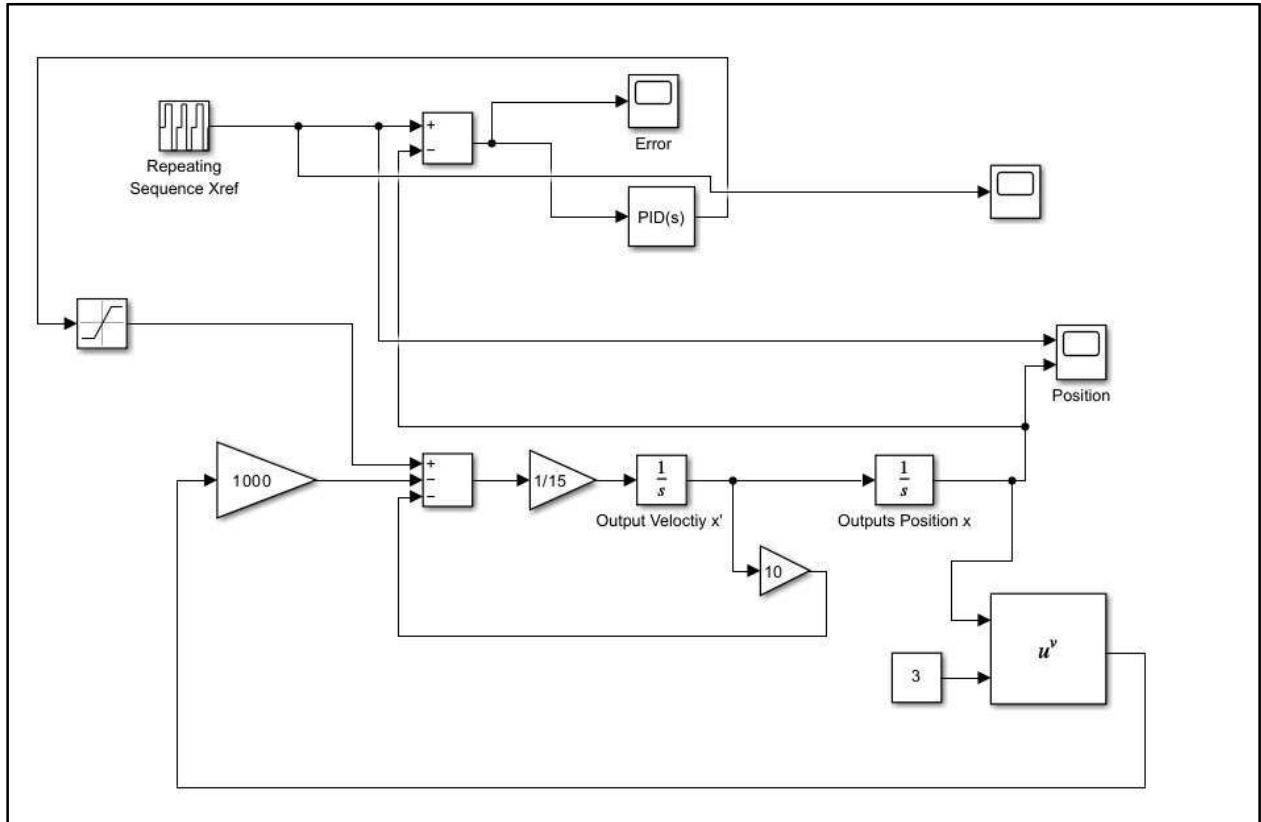


Figure 12: Nonlinear System and the PID Controller

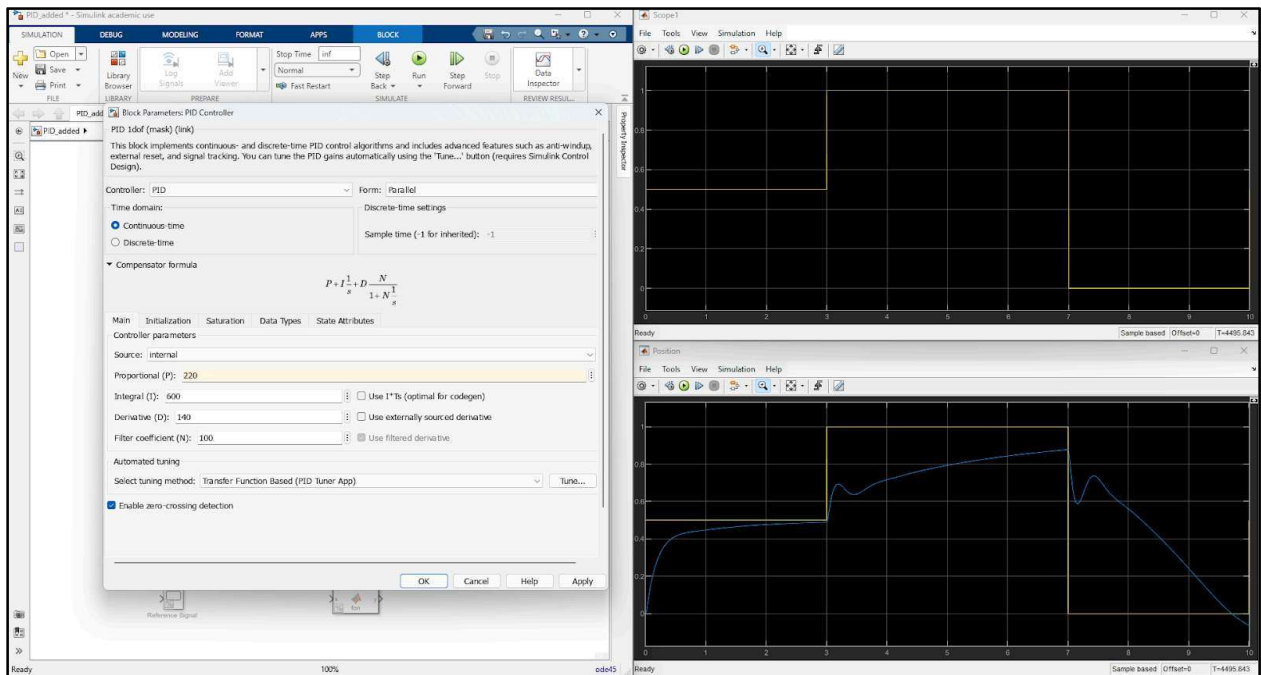


Figure 13: Overall System, Gains, and Outputs

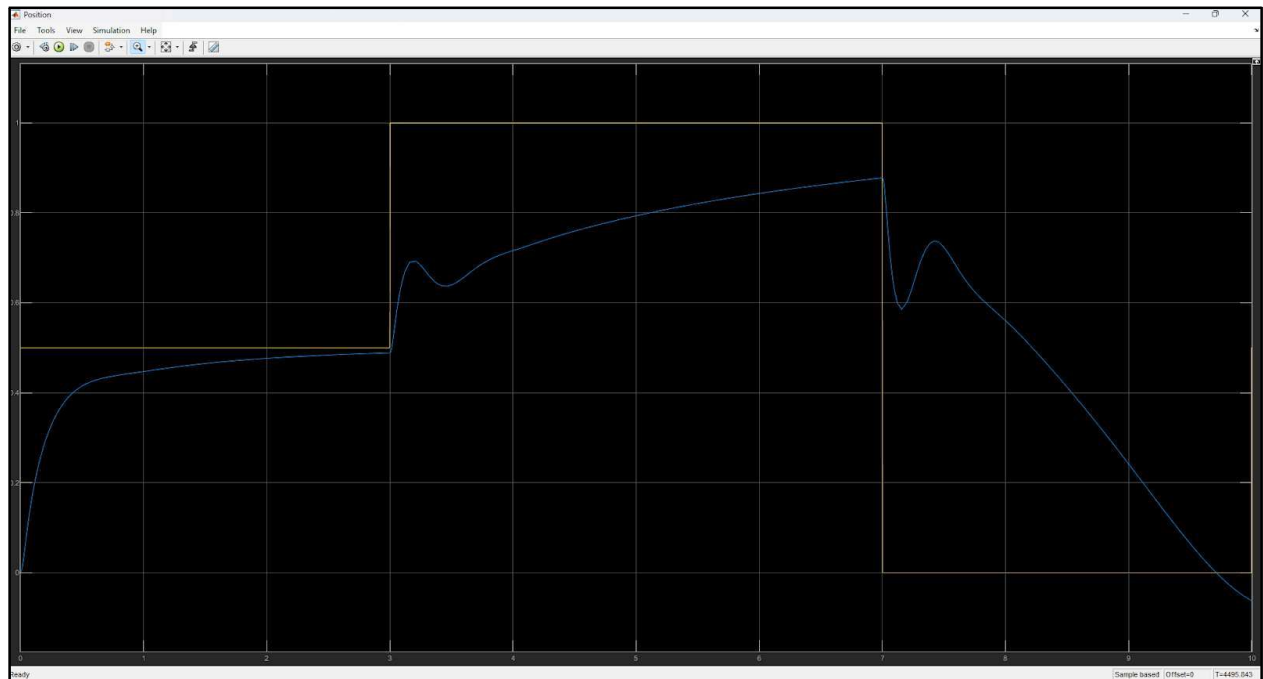


Figure 14: Position Output Measured vs Desired

From the graph above, the speed response is slower than desired, so K_p needs to increase, K_i needs some adjusting to reduce the steady state error. Some oscillation and overshoot are present, which means K_d will need more tuning. The PID controller was designed using empirical methods, and through a combination of trial-and-error adjustments and PID tuning techniques, we refined the gains to produce an output that closely followed the desired shape of the reference signal.

$K_p=5000$, $K_i=5000$, and $K_d=1000$

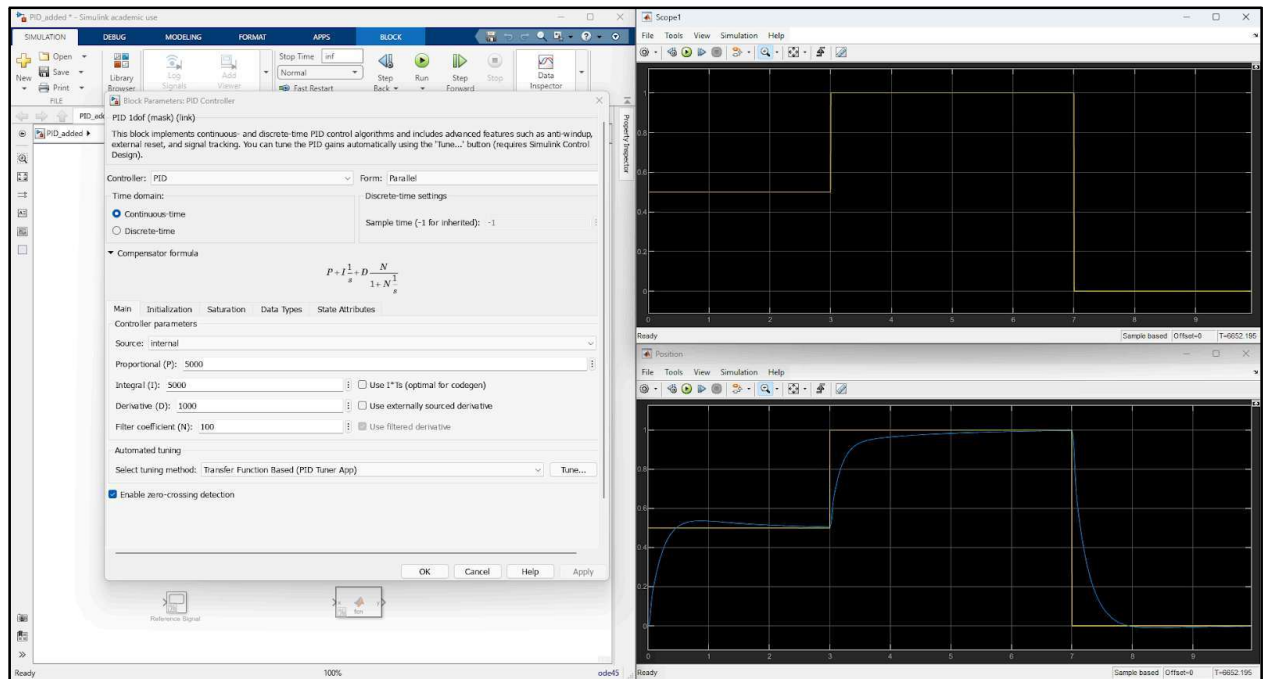


Figure 15: Overall System, Gains, and Outputs Run 2

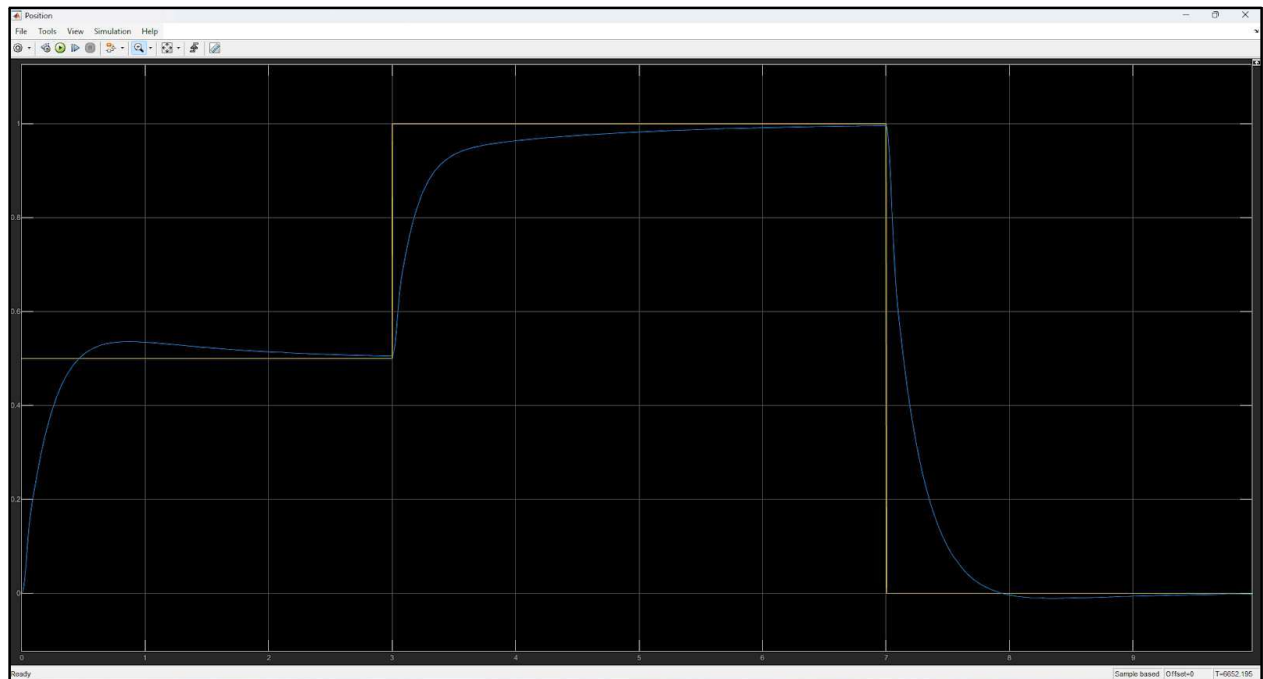


Figure 16: Position Output Measured vs Desired Run 2

$K_p=5000$, $K_i=5450$, and $K_d=1250$

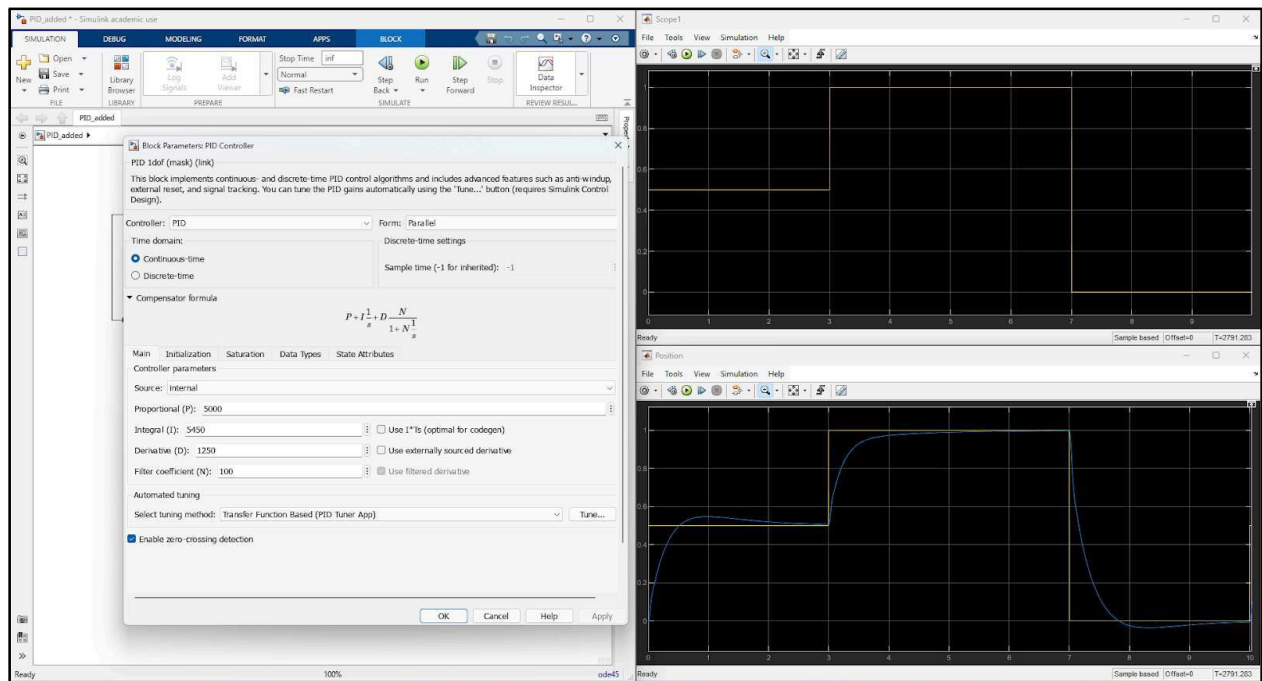


Figure 17: Overall System, Gains, and Outputs Run 3

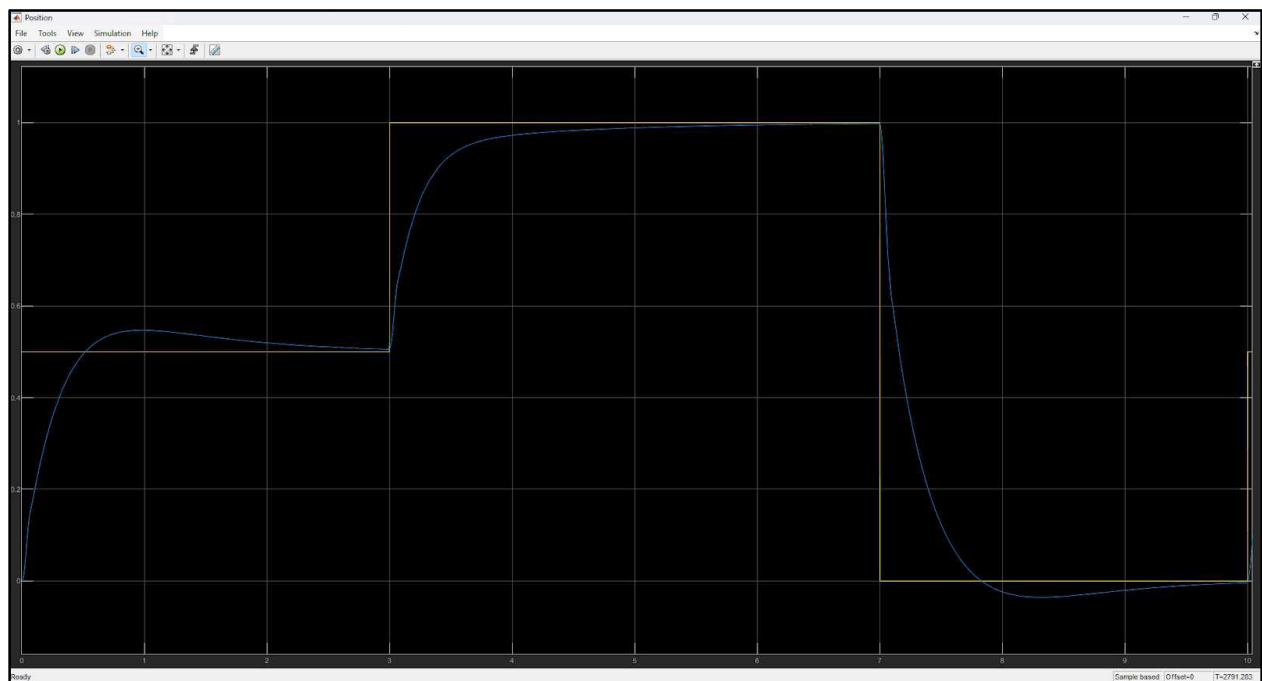


Figure 18: Position Output Measured vs Desired Run 3

The control gains used for the PID tuning to create the final result are quite high and require trial and error in the final stage to get closer to the reference signal.

The final gains are 5000, 5450, and 1250 for k_p , K_i , and K_d , respectively. While this tuning produces a result that more closely matches the desired shape, it also causes the output to oscillate more, leading to a less stable response.

c) In real world applications, due to the constraints for actuators inputs (e.g. saturations) the controller parameters (k_p , k_D and k_i) are not allowed to be in arbitrary range. Try to design a new set of controller parameters (k_p , k_D and k_i), such that they satisfy the following conditions.

$$0 < k_p \leq 800, 0 < k_D \leq 250, 0 < k_i \leq 1800$$

Design Specifications:

- Avoid overshoots.
- The system must be as fast as possible (the system should reach the desired value in less than three seconds).
- It is allowed to have less than 2% error in three seconds (i.e. settling time $\leq 3 \text{ sec}$).

1. $K_p=800$, $K_i=1800$, and $K_d=250$

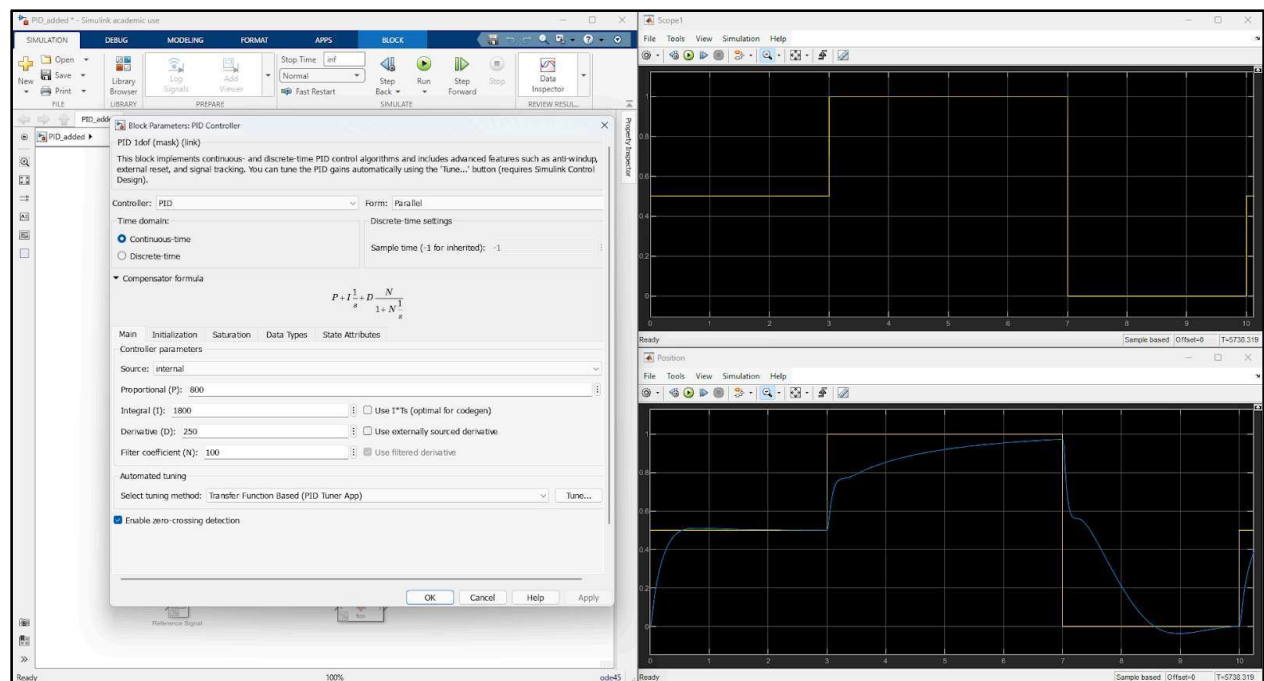


Figure 19: Overall System, Gains, and Outputs Constraint Run

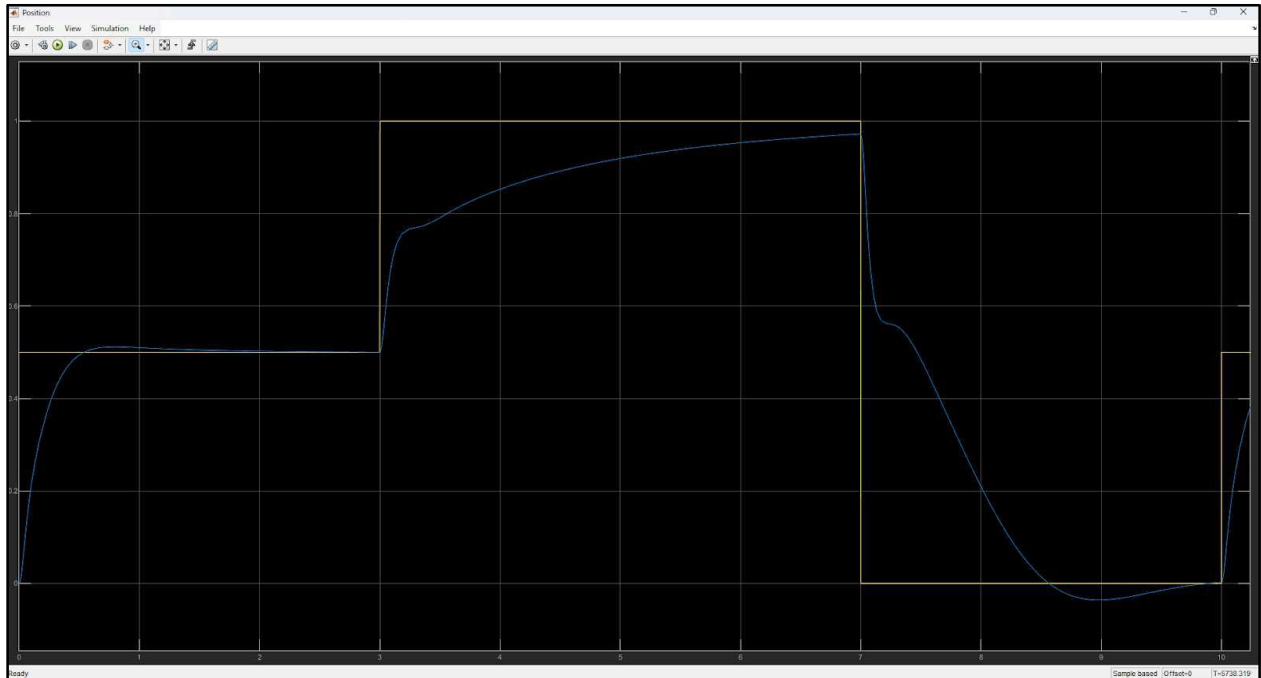


Figure 20: Position Output Measured vs Desired Constraint Run

2. $K_p=5000$, $K_i=5450$, and $K_d=250$

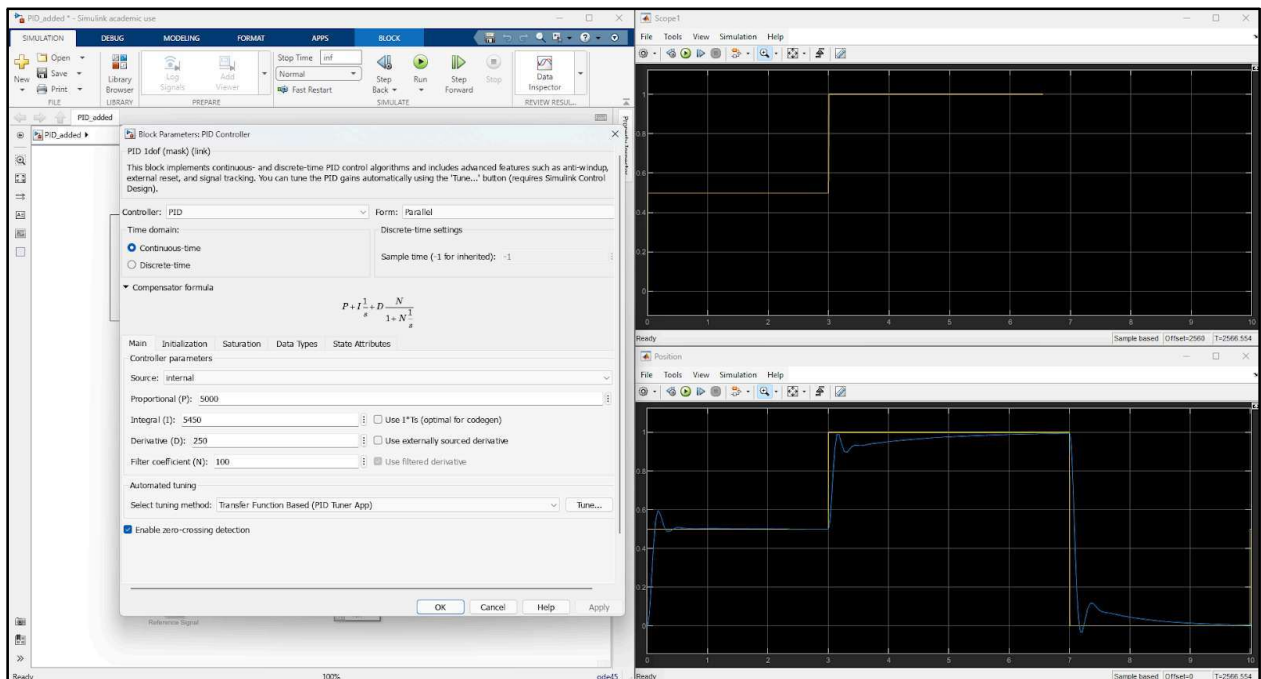


Figure 21: Overall System, Gains, and Outputs $K_d=250$

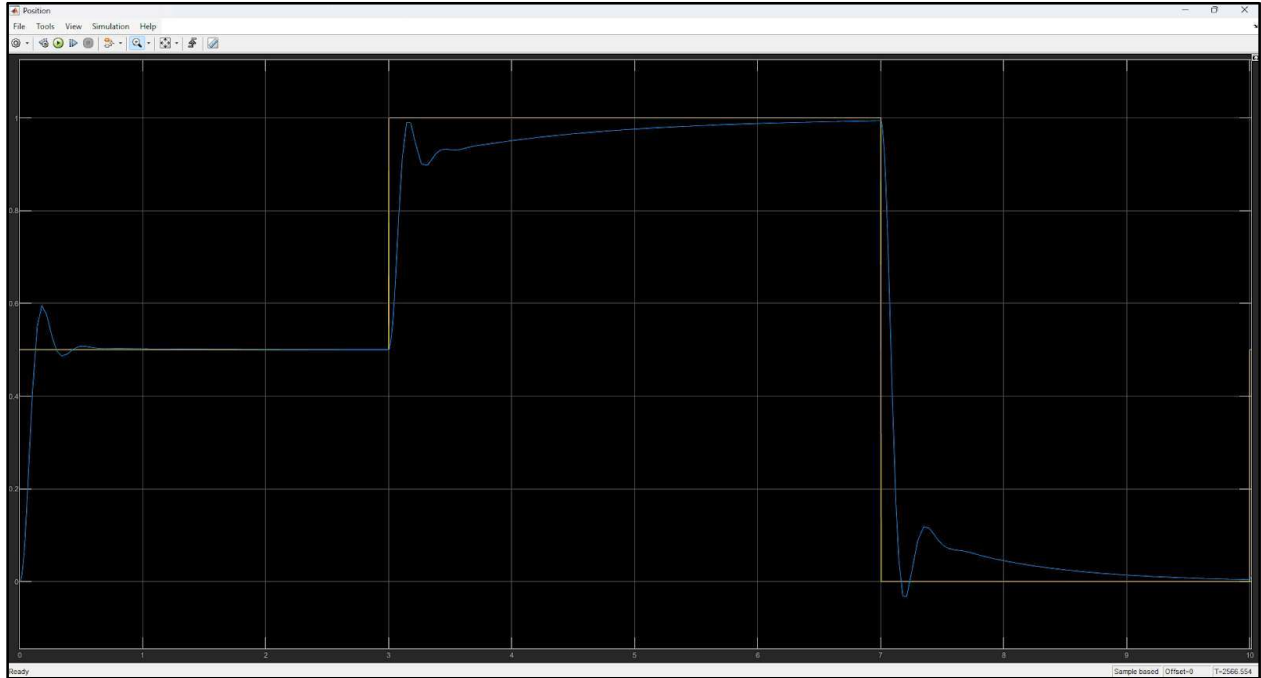


Figure 22: Position Output Measured vs Desired $K_d=250$

Steady State response is 1.21% away from the desired results at $x_0=1$

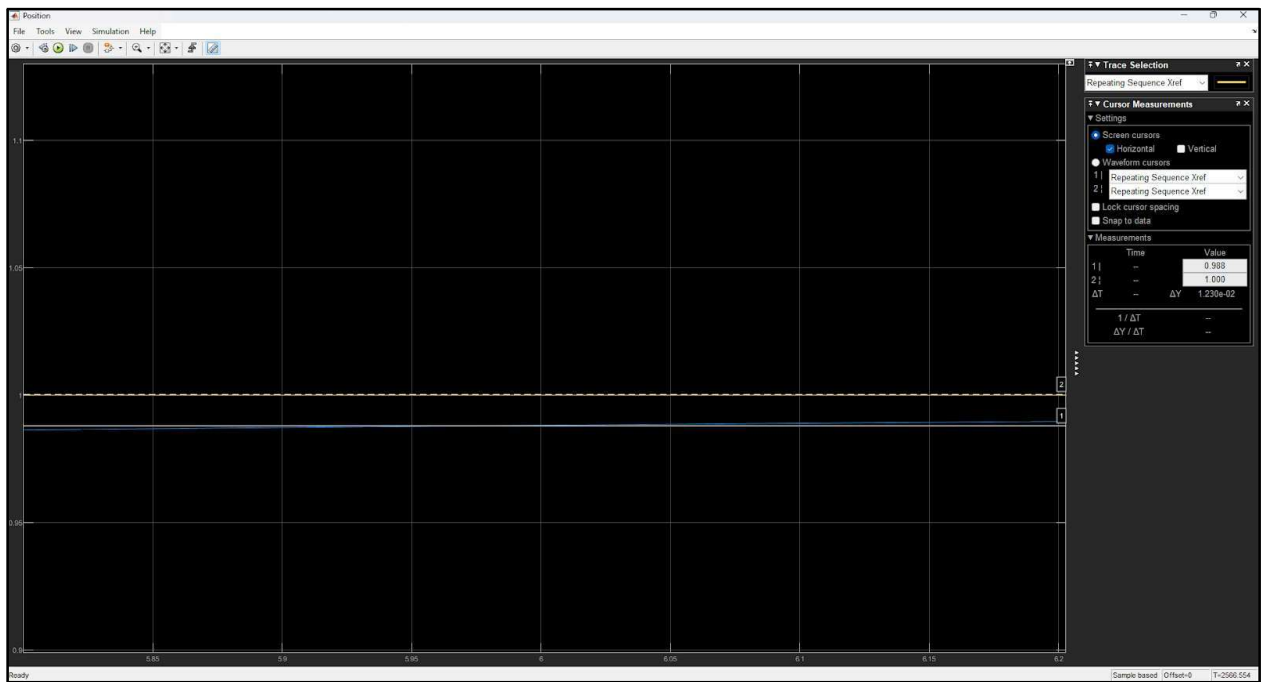


Figure 23: Steady State at $x_0=1$

Steady State response at $x_0=0$

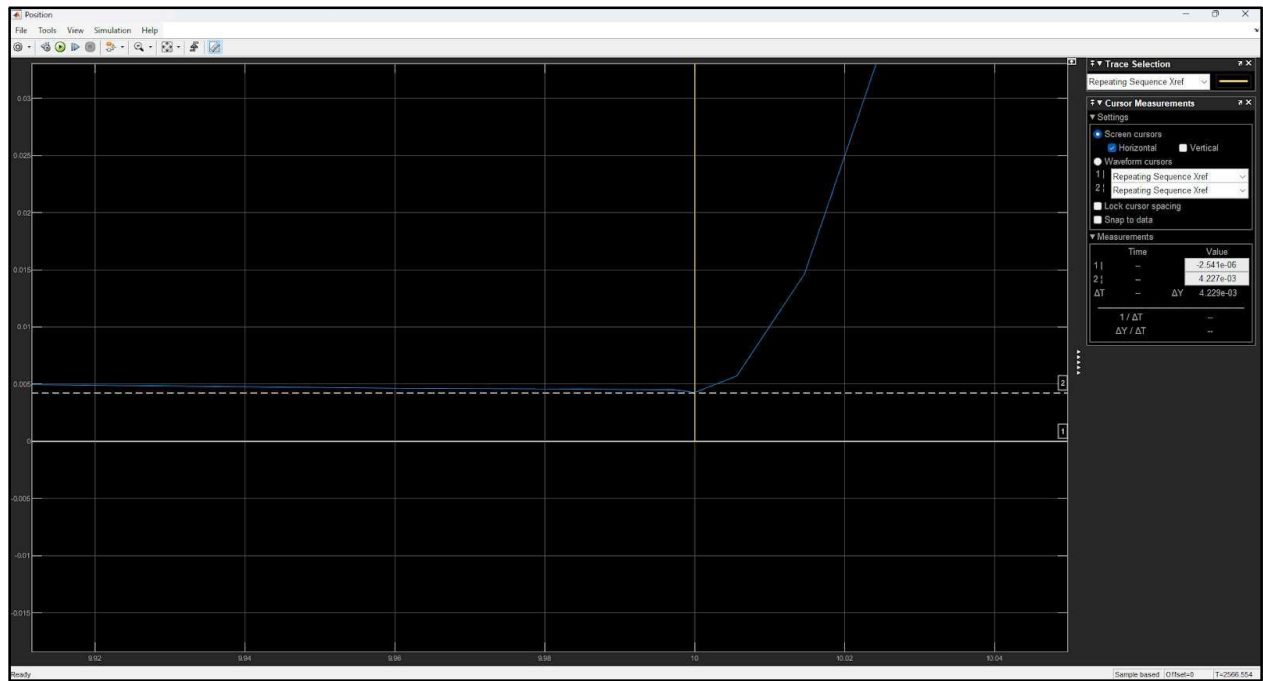


Figure 24: Steady State at $x_0=0$

3. Intro to Gain-Scheduled PID Controller

A gain-scheduled PID controller was designed to improve performance over the fixed gain in part 2. The controller has to track the reference signal, from Figure 3 in the final project document, with a settling time of less than 3s, and less than 2% steady-state error, minimal overshoot, and stay within the limits for the gains.

The gains of the gain-scheduled PID controller change, depending on the system's operating point. Divided into subregions, the gains, K_p , K_i , and K_d , were separated into each subregion. The error signal was used as the input for the three lookup tables (LUTs), which provided proportional, integral, and derivative gains to the PID controller in Simulink.

Below is the Simulink model used for part 3.

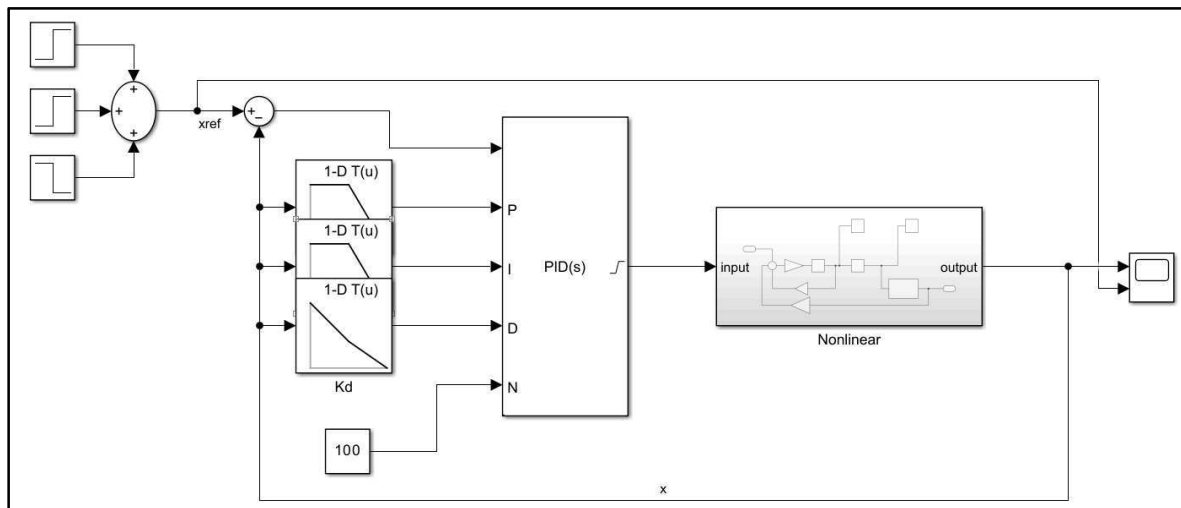


Figure 25: Simulink Model with Lookup Tables

Setting the gain as follows:

$$K_p = [310 \ 290 \ 310]$$

$$K_i = [290 \ 270 \ 160]$$

$$K_d = [250 \ 240 \ 250]$$

$$N = 10000$$

Below is the response for this section.

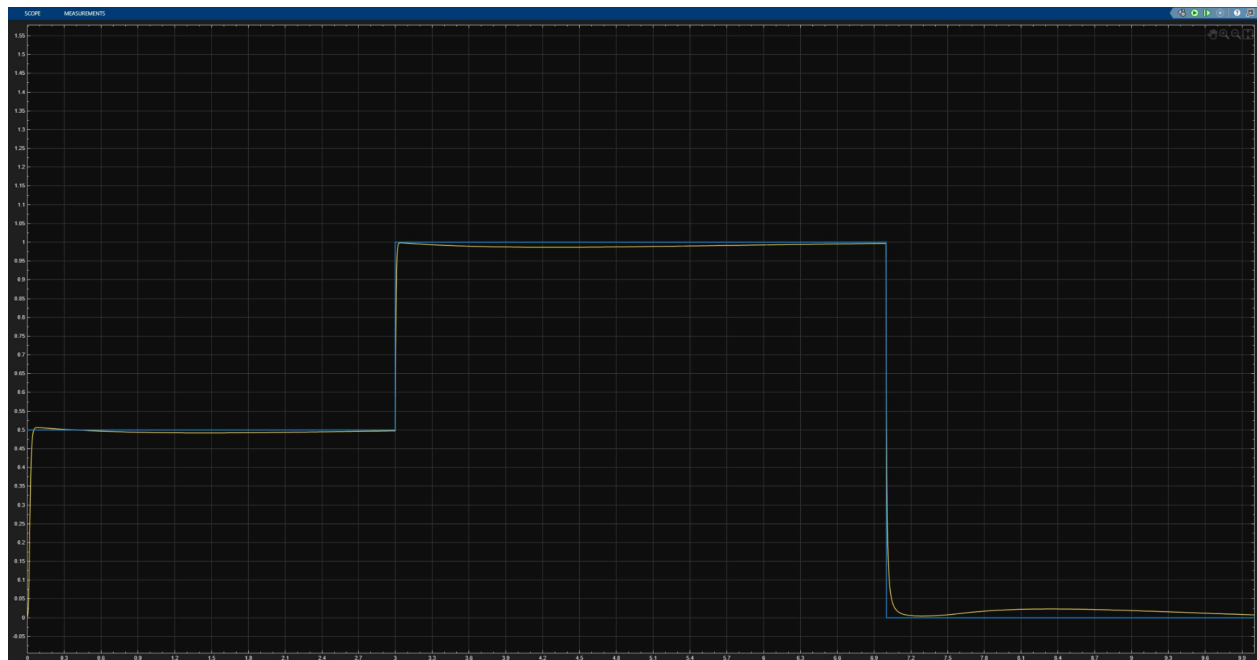


Figure 26: Response for Gain-Scheduled PID Controller

At $x_0=0.5$, the difference is only 0.624%, where the measured value was at 0.49688m instead of 0.5m



Figure 27: Close-Up of Response @ X = 1

The plot above compares the commanded door position (in blue) and the measured position (in yellow) under a gain-scheduled PID. Our controller successfully met the design requirements by maintaining a steady-state error within 2% for the reference position of $x_0 = 0.5m$ in the required 3-second settling time, as seen in the scope response. The value stabilized at 0.49688m, indicating only a 0.00312 m difference from the desired setpoint and showed minimal overshoot and no violations of the gain constraints. Our controller adjusts K_p , K_d , K_i dynamically across the subregions based on the system's operating point, improving the tracking performance compared to the fixed-gain PID design from section 2. This demonstrates the advantages of gain scheduling in handling varying operating conditions while maintaining precision and stability.

4. Gain-Scheduled PID Controller Design, Simulation & Analysis

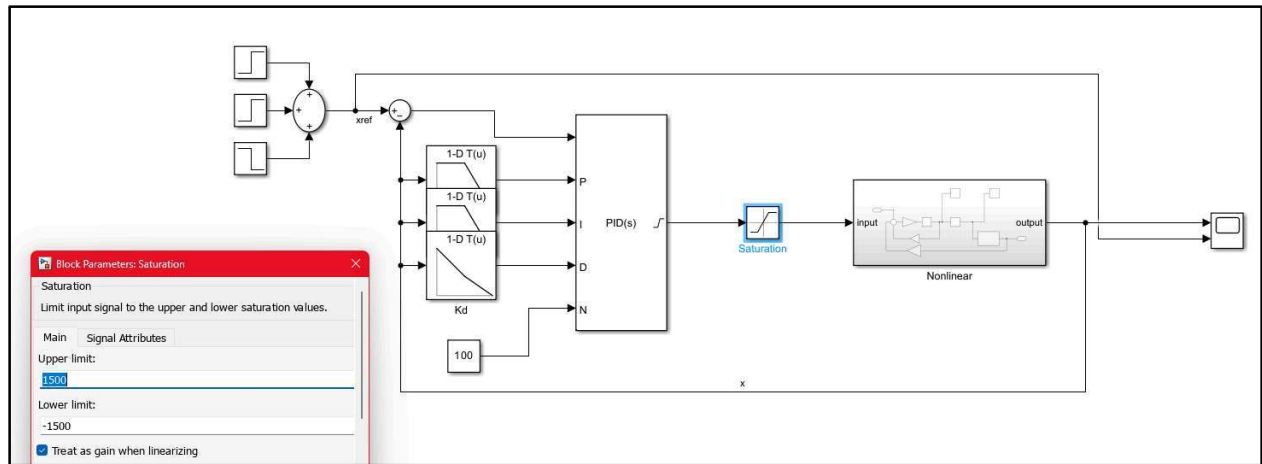


Figure 28: Simulink Model for Gain-Scheduled PID with Saturation

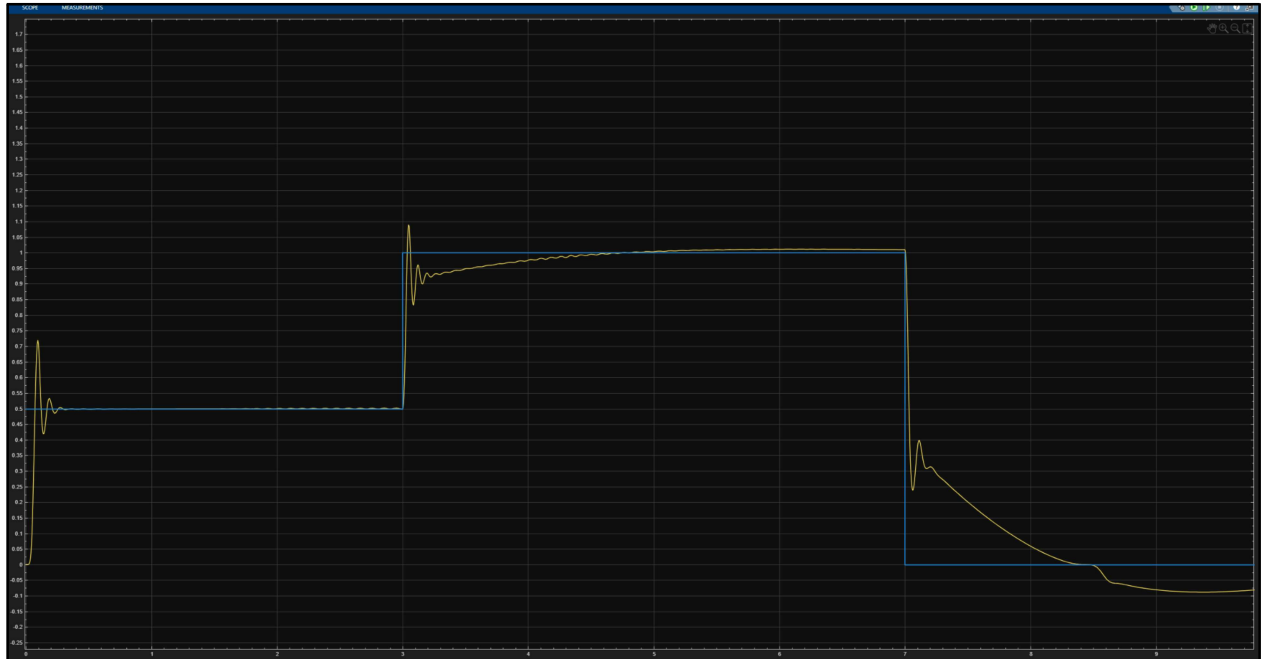


Figure 29: Response for Part 4

In Part 3, the gain-scheduled PID tracks the reference almost perfectly because the controller can apply whatever force it computes. When we impose the actuator limit with a Simulink Saturation block (± 1500 N) and keep the same gains, the response changes noticeably. Around each step, the commanded force is clipped at the limit, so the loop behaves as if its effective gain were reduced. While the output is saturated, the error cannot be driven down quickly, and the integrator accumulates. When the signal comes out of saturation, the stored integral produces the transient overshoot and ringing seen after the 0.5 to 1 m step, and the undershoot after the 1 to 0m step. In short, the saturation introduces a nonlinearity that slows the rise, reduces phase margin, and can create integral windup during the clipped intervals. We did not retune the controller; the plot therefore illustrates the practical trade-off of when a realistic force limit is enforced, performance is less glued to the reference but remains stable and convergent. In a real design, we would mitigate this by enabling PID anti-windup, slightly reducing, adding a small set-point filter/rate limit, and/or increasing near the open end to damp the post-saturation transient.

The response in Figure 29 at $X = 0.5$

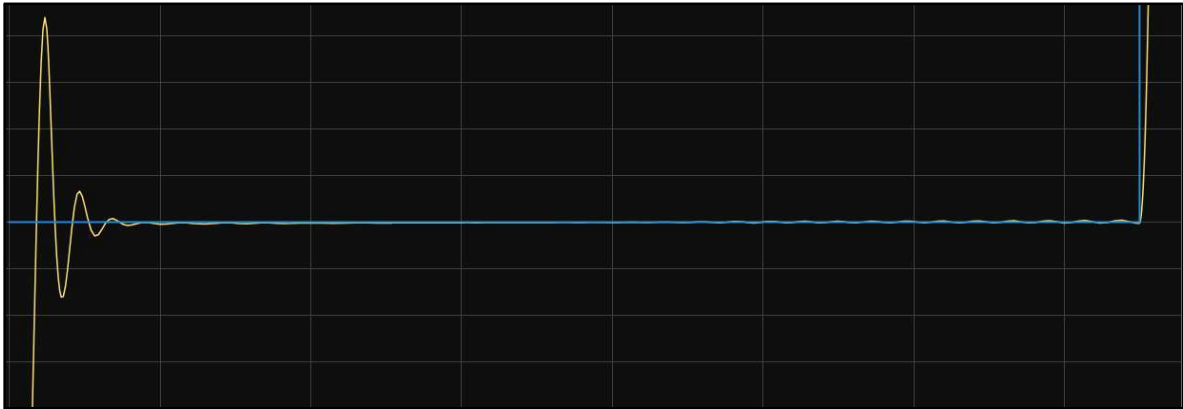


Figure 30: Close-Up Response at $X = 0.5$

The response in Figure 29 at $X = 1$

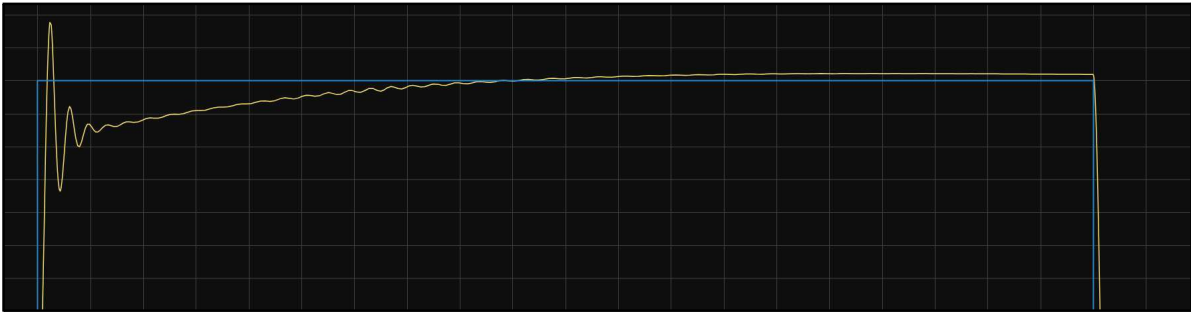


Figure 31: Close-Up Response at $X = 1$

The response in Figure 29 at $X = 0$

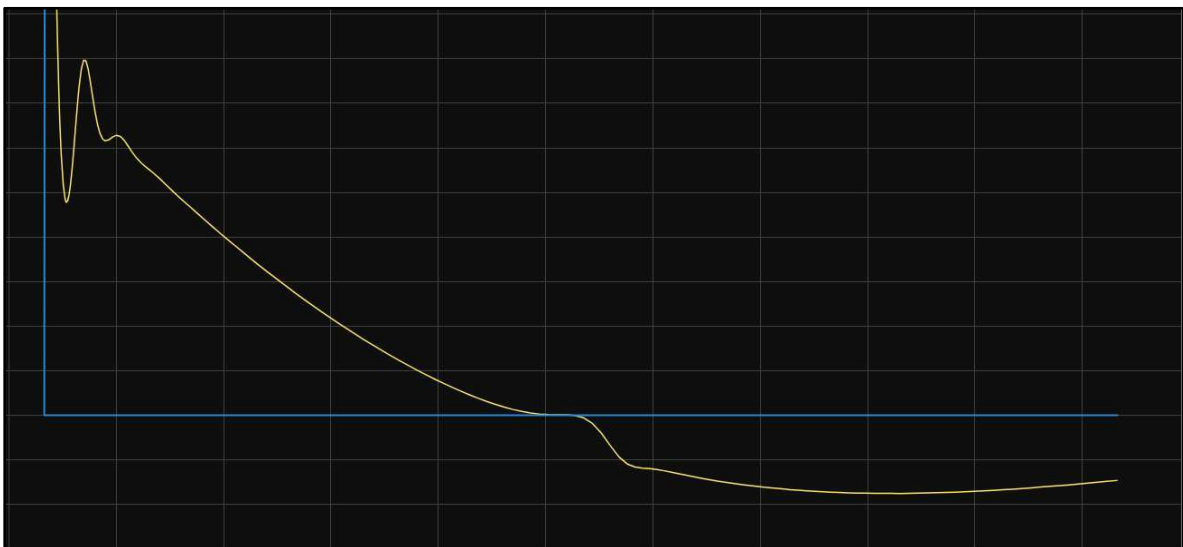


Figure 32: Close-Up Response at $X = 0$

Discussion

The gain-scheduled PID controller was implemented in Simulink with different operating subregions, each has their own K_p , K_i , and K_d values. The design allowed the controller gains to adjust based on the system's position, where the ultimate goal was to improve the performance compared to a regular fixed-gain PID. The gains were scheduled across the subregions with lookup tables (LUTs), driven by the system's operating point. This allows the controller to adapt dynamically. It improved the tracking precision in mid-range positions and maintained stability without overshoot. The saturation block after the PID output ensured that the actuator limits were there and that the control signals show no clipping for the chosen gains.

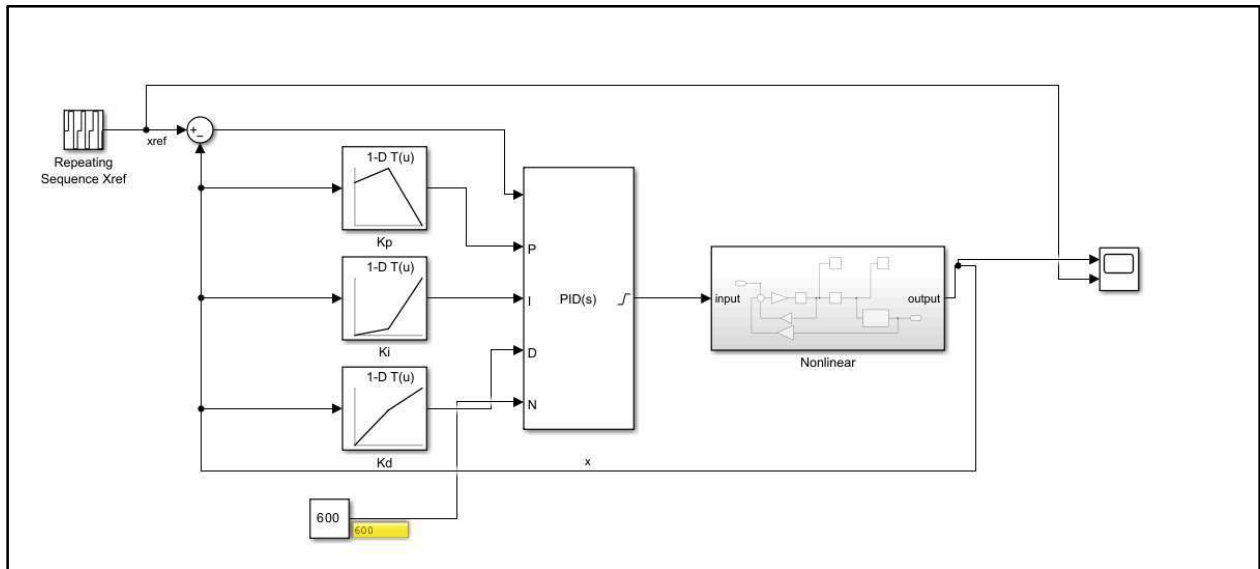
The controller consistently achieved steady-state error within 2% and settling times under 3s for reference points like $x = 0.5\text{m}$. Even though performance in certain ranges could be further refined with more tuning, the implemented approach demonstrated that gain scheduling can significantly improve tracking performance under varying conditions compared to a single fixed gain set.

Conclusion

Overall, the objective of designing and analyzing a gain-controlled PID controller was successfully implemented to meet the project's performance and constraint requirements. The controller maintained less than 2% steady-state error, minimal overshoot, and achieved settling times under 3s without exceeding actuator constraints. Scheduling gains by operating point helped improve response over the fixed-gain design from Part 2, especially in the mid-range, while still maintaining overall stability. This project highlighted the benefit of gain scheduling in nonlinear systems, especially under actuator constraints. Further improvements could include more subregions and better tuning to further optimize the performance.

Appendix

Using Repeating Sequence



Setting up the gains as follows:

$$K_p = [700 \ 800 \ 400]$$

$$K_i = [100 \ 290 \ 1800]$$

$$K_d = [100 \ 190 \ 250]$$

